Connect Four

Bas Terwijn

February 15, 2026

1 Introductie

Dit document behandelt de Connect Four game met heuristiek.

1.1 Doelen

Bekend raken met:

- Verschillende implementatie van het `Game_Abstract` interface
- Implementatie van heuristiek voor Connect Four

2 Connect Four

Naast een `Tictactoe` implementatie kunnen we ook een `Connect_Four` (Nederlands: Vier-op-een-rij) implementatie schrijven voor het `Game_Abstract` interface zoals weergegeven in figuur 1. Omdat elke `Player` alleen gebruik van dit `Game_Abstract` interface, en niets weet van de implementatie van een specifieke game, werkt elke `Player` ook voor deze nieuwe `Connect_Four` game.

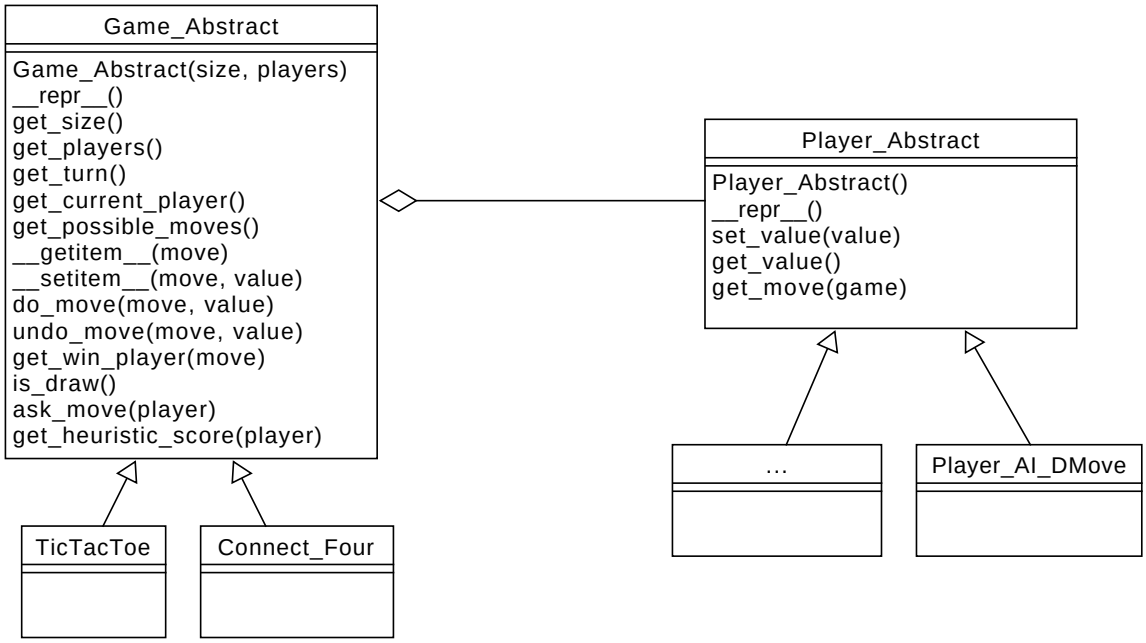

Figure 1: Connect Four als tweede Game

Deze `Connect_Four` in file `connect_four.py` is al geïmplementeerd. Lees de implementatie door. Om als mens een `Connect_Four` spel te spelen tegen een `Player_AI_DMove` speler die 6 zetten vooruit denkt runnen we:

```
$ python play.py -g c4 -p1 h -p2 d6
```

Het is al best moeilijk om te winnen van deze `Player_AI_DMove` speler, al maakt deze nog wel rare zetten omdat deze nog geen heuristiek gebruikt. Alle mogelijke zetten die niet binnen 6 zetten tot winst of verlies leiden krijgen nu een `Score(0, 0, 0)` score, waardoor de speler willekeurig kiest tussen deze zetten. Dat leidt soms tot een zet die niet erg intelligent lijkt. Het zou beter zijn om ook voor `Connect_Four` een heuristiek te gebruiken.

2.1 Opdracht `connect_four_evaluator.py`

Om een heuristiek te maken voor `Connect_Four` kiezen we hier om te kijken naar alle mogelijke winnende lijnen van lengte vier of meer op het bord. Lijnen kunnen horizontaal, verticaal of diagonaal zijn. Voor elke lijn bepalen we de score en alle scores bij elkaar opgeteld is de waarde van de heuristiek. De code hiervoor is te vinden in file `connect_four_evaluator.py`, maar wat nog ontbreekt is de implementatie van de `Connect_Four_Evaluator` class die de score voor een lijn berekent. Deze class heeft drie methods die door u moeten worden geïmplementeerd:

- `def __init__(self, win_length, empty_value, my_value):`
- `def clear(self):`
- `def add(self, value):`

Gebruik deze test-code om deze class te testen:

```
$ pytest test_connect_four_evaluator.py -vv
```

De eerste test in deze test-code die de heuristische scores van een bepaalde lijn berekent, is deze:

```
evaluator = Connect_Four_Evaluator(4, '-', 'X')
values = ['X', 'X', '-', 'X', 'X', '-', 'X', '-', '-', 'X', '-', '-', 'X']
expected_scores = [0, 0, 0, 3, 3, 2, 3, 2, 1, 2, 1, 1, 2]
```

Elke keer als de volgende value van een lijn wordt toegevoegd met `evaluator.add(value)`, wordt de score berekend voor een speler, in dit voorbeeld speler 'X'. De score is 0 als de speler niet kan winnen op de laatste `win_length=4` toegevoegde values. De speler kan winnen als er al `win_length` values zijn toegevoegd en elke van de laatste `win_length` values van deze speler is of nog leeg is. De score is dan het aantal values van deze speler. Voor de eerste test is dat hier schematisch weergegeven waarbij elke regel overeenkomt met toevoegen van de volgende value met `evaluator.add(value)` en waarmee met `---` wordt aangegeven wat de laatste `win_length` values zijn:

```
values = ['X', 'X', '-', 'X', 'X', '-', 'X', '-', '-', 'X', '-', '-', 'X']
          --- 0
          -------- 0
          -------------- 0
          ------------------- 3
               ------------------- 3
                    ------------------- 2
                         ------------------- 3
                              ------------------- 2
                                   ------------------- 1
                                        ------------------- 2
                                             ------------------- 1
                                                  ------------------- 1
                                                       ------------------- 2
```

Als de laatste `win_length` values een mix van speler- en tegenspeler-values is, dan kan daar niet meer gewonnen worden en is de score daar 0. Als de laatste `win_length` values alleen tegenspeler values of lege values zijn, dan is de score de negatieve waarde van aantal values value van de tegenspeler zoals hier schematisch weergegeven:

```
values = ['X', 'X', 'X', 'X', 'O', 'O', 'O', 'O']
          --- 0
          -------- 0
          -------------- 0
          ------------------- 4
               ------------------- 0
                    ------------------- 0
                         ------------------- 0
                              ------------------- -4
```

Implementeer de `Connect_Four_Evaluator` class in file `connect_four_evaluator.py` en test uw implementatie met de eerder genoemde test-code. Het kan voor de implementatie nuttig zijn om gebruik te maken van de deque class uit de `collections` module om de laatste `win_length` values bij te houden. Als ook de `clear()` method goed geïmplementeerd is, zodat een `Connect_Four_Evaluator` object hergebruikt kan worden voor verschillende lijnen, dan zouden ook de tests voor de `Connect_Four_Heuristic` class moeten slagen:

```
pytest test_connect_four_heuristic.py -vv
```

3 Connect Four met Heuristiek

Wanneer de `Connect_Four_Evaluator` class is geïmplementeerd, dan kunnen we tegen een `Player_AI_DMove` speler met heuristiek spelen en zal de speler een stuk intelligenter lijken. Maar, omdat het berekenen van de heuristiek wel extra tijd kost, moeten we de diepte van de zoekboom wat verlagen om binnen een redelijke tijd een

zet te kunnen kiezen. Probeer bijvoorbeeld te spelen tegen een `Player_AI_DMove` speler die 5 zetten vooruit denkt met heuristiek:

```
$ python play.py -g c4 -p1 h -p2 d5h
```

Welke speler zou vaker winnen, een speler die 6 zetten vooruit denkt zonder heuristiek, of een speler die 5 zetten vooruit denkt met heuristiek?

```
$ python play.py -g c4 -p1 d6 -p2 d5h
```

Om een sterkere `Player_AI_DMove` speler te krijgen zouden we kunnen experimenteren met verschillende manieren om een heuristische waarde van een bord te berekenen, maar daar hebben we nu geen tijd voor.